COMPILERS · PARSING

Development of an LL(1) Parser

FIRST, FOLLOW, parsing table generation, and predictive parsing.

Team: 2420030569 RANJJETH · 2420030256 SRUJAN · 2420030315 AKHIL · 2420030061 DRONAVALLI SRI GOWTHAM



Made with GAMMA

THE PROBLEM

CHOOSE ONCE.

Why predictive parsing?

A predictive parser connects a grammar to a parser that chooses productions without backtracking.



LL(1)

Scan input left to right, build a leftmost derivation, and use one lookahead token.



The payoff

Simple control flow and fast error detection for suitable grammars.

Made with GAMMA

START HERE

Start with an LL(1)-friendly grammar

Use a small expression grammar to make every later step visible; prepare it before computing any sets.

EXPRESSION GRAMMAR

$$E \rightarrow T\ E'$$
$$E' \rightarrow +\ T\ E' \mid E$$
$$T \rightarrow ID$$

Why this shape works

Removing left recursion and applying left factoring can make predictive decisions possible. Here, each production begins with a distinct clue: **id**, **+**, or the empty string.

Made with GAMMA

SET 01

FIRST answers: what can begin here?

FIRST(α) contains terminals that can begin strings derived from α.

Iterative rule: inspect symbols left to right, and continue only when ε remains possible.

01

Start with the first symbol

FIRST(id) = { id }

02

Continue if ε is possible

FIRST(+ T E′) = { + }

03

Keep the empty option

Because E′ can vanish, FIRST(E′) = { +, ε }.

Made with GAMMA

SET 02

FOLLOW answers: what can come after?

FOLLOW(A) contains terminals that may appear immediately after nonterminal A.

FOLLOW is about context, not beginnings.

01

Mark the start

Place $ in FOLLOW(E) because E is the start symbol.

02

Read the production

From E → T E′, add FIRST(E′) − {ε} to FOLLOW(T).

03

Propagate when it vanishes

Because E′ can disappear, add FOLLOW(E) to FOLLOW(T) and FOLLOW(E′).

FOLLOW carries the valid next token when a nonterminal finishes, including the end marker $.

Made with GAMMA

TABLE

Turn the sets into a parsing table

Each table cell answers one question: with nonterminal **A** on the stack and lookahead **a**, which production applies?

NONTERMINAL	ID	+	$
E	E → T E′	—	—
E′	—	E′ → + T E′	E′ → ε
T	T → id	—	—

For A → α, place it in M[A, a] for every a in FIRST(α). If ε is in FIRST(α), use every b in FOLLOW(A).

No cell may contain two productions in an LL(1) grammar.

Made with GAMMA

RUNTIME

How the table-driven parser runs

01

Initialize

Stack: $ and the start symbol. Input ends with $.

02

Match terminals

Match the stack top with lookahead, or report an error.

03

Expand nonterminals

Consult M[top, lookahead], push the right-

One trace: input **id + id $**

STACK TOP	LOOKAHEAD	ACTION
E	id	Use E → T E′
T	id	Use T → id
id	id	Match
E′	+	Use E′ → + T E′
E′	$	Use E′ → ε

The parser alternates between table lookup and terminal matching.

Made with GAMMA

TAKEAWAY

GRAMMAR → TABLE → TRACE

Build the pipeline, then make each stage observable.

Implementation checklist and takeaway

01

Represent

Productions, terminals, nonterminals, FIRST/FOLLOW sets, and table entries.

02

Compute

Iterate sets to a fixed point and detect table conflicts.

03

Trace

Show stack and input together, with clear syntax errors.

Team deliverable: a working parser with readable traces and clear syntax errors.

Made with GAMMA